



Umm Al Qura University
Software Engineering Department
College of Computing

Wejha

A project submitted

*in partial fulfillment of the requirements for the degree of
Bachelor in Software Engineering*

by

Suhail Maher Kafyah (444005606)

Fares Ahmed Talegdar (444004699)

Ibrahim Salah Hassan (444004379)

Mohammed Adel Alghamdi (444004542)

Supervised by

Supervisor's Dr. Abdulhadi Eidroos

Declaration

This is to declare that the project entitled "WEJHA — Smart Stadium Navigation Application" is an original work done by the undersigned, in partial fulfillment of the requirements for the degree "Bachelor in Software Engineering" at the Software Engineering Department, College of Computing, Umm Al-Qura University.

All the analysis, design, and system development have been accomplished by the undersigned. This project has not been submitted to any other college or university.

Suhail Maher Kafyah _____

Fares Ahmed Talegdar _____

Ibrahim Salah Hassan _____

Mohammed Adel Alghamdi _____

ABSTRACT

WEJHA is a smart mobile application designed to improve the experience of football fans inside stadiums by providing intelligent seat navigation and crowd-aware routing. During large sporting events, many attendees face difficulties locating the correct gate, section, and seat due to overcrowding and unfamiliar stadium layouts. The proposed system addresses these challenges by guiding users from the entrance gate to their assigned seat through the best available route.

The application is initially designed for King Abdullah Sports City Stadium (Alinma Stadium) and can later be expanded to support multiple stadiums. WEJHA is developed as a cross-platform mobile application using Flutter for the frontend and Supabase as the backend platform. The system includes ticket integration, indoor stadium maps, static and dynamic navigation, real-time crowd monitoring, notifications, and an administrative dashboard for stadium management.

The project aims to reduce congestion, minimize time loss, improve fan safety, and enhance the overall stadium experience. Future improvements may include AI-based crowd prediction, live tracking, and integration with official ticketing platforms

ACKNOWLEDGMENT

The team would like to express sincere gratitude to the faculty of the Software Engineering Department at Umm Al-Qura University for their continuous guidance throughout this project. We also extend appreciation to our families and to the football fans who participated in our requirements-gathering interviews, whose real-world insights directly shaped the design of WEJHA.

CONTENT

Declaration	1
ABSTRACT	2
ACKNOWLEDGMENT	3
CHAPTER 1: INTRODUCTION	9
1.1 Overview	9
1.2 Project Motivation	10
1.3 Problem Statement	11
1.4 Project Aim and Objectives	11
1.4.1 Project Aim	11
1.4.2 Project Objectives	11
1.5 Related Existing Systems	12
CHAPTER 2: PLANNING PHASE	13
2.1 Scope of the project	13
2.1.1 Included Features	13
2.1.2 Excluded Features	13
2.2 Project risks and Product risks	14
2.3 Feasibility study	15
2.3.1 Technical Feasibility	15
2.3.2 Schedule Feasibility	15
2.3.3 Operational Feasibility	15
2.4 Project Schedule	16
2.5 Project Software and Hardware Requirements	16
2.5.1 Software Requirements	16
2.5.2 Hardware Requirements	16
CHAPTER 3: REQUIREMENT ENGINEERING AND ANALYSIS	18
3.1 Used Techniques For Requirements Collection	18
3.2 Functional Requirement & Modelling	18
3.2.1 List of System Functions (Features)	18
User Features	18
Admin Features	19
3.2.2 Use Case Diagram	19
3.2.3 Use Cases: Description & Details	21
Use Case: Navigate to Seat	21
Actor	21
Preconditions	21
Main Flow	21
Alternative Flow	21

3.4 Nonfunctional Requirements: Quality & Constraints_____	21
CHAPTER 4: ARCHITECTURE & DESIGN_____	23
4.1 Software Architecture_____	23
4.2 Software Detailed Design_____	24
4.2.1 Use Cases Internal Interactions as Sequence Diagrams_____	24
4.2.2 Class Diagram_____	31
4.2.3 Data Storage Organization_____	32
5.1 Tools_____	34
5.2 System Testing(Black Box)_____	34
5.3 Unit Testing(White Box)_____	35
CONCLUSIONS_____	37
REFERENCES_____	38
APPENDIX-A: USER MANUAL (ONLY FOR PROJECT 2)_____	39

LIST OF TABLES

Table 1.1 : Related Existing Systems	13
Table 2.1 : Project risks and Product risks	16
Table 2.1 : List of System Functions	16
Table 3.1 : Nonfunctional Requirements	23
Table 5.1 : Tools and frameworks	35
Table 5.2 : Unit tests	43
Table 5.3 : System tests	46
Table 5.4 : User Acceptance test	49

LIST OF FIGURES

Figure 2.1 : Context Diagram_____	15
Figure 2.1: Gantt Chart_____	17
Figure 3.1 : Use cases_____	21
Figure 4.1 : Simplified Architecture_____	29
Figure 4.2 :Sequence diagram of User Registration_____	30
Figure 4.3 : Sequence diagram of User Login_____	31
Figure 4.4 : Sequence diagram of View Ticket & QR Code_____	32
Figure 4.5 : Sequence diagram of Navigate to Seat_____	33
Figure 4.6 : Sequence diagram of Update Gate Status & Send Warning_____	34
Figure 4.7 : Chatbot interaction_____	35
Figure 4.8 : Sequence diagram of Real-time Crowded Monitoring_____	36
Figure 4.9 : Class Diagram_____	37
Figure 4.10 : ERD Data Organization_____	38
Figure 4.11 : User Screens_____	39

LIST OF ACRONYMS AND ABBREVIATIONS

Acronym	Full Form
API	Application Programming Interface
UI	User Interface
UX	User Experience
DB	Database
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case
QR	Quick Response (Code)
SDK	Software Development Kit
REST	Representational State Transfer
MVVM	Model-View-ViewModel
GPS	Global Positioning System
AI	Artificial Intelligence
LLM	Large Language Model
OTP	One-Time Password
RTL	Right-to-Left (text direction)

CHAPTER 1: INTRODUCTION

This chapter comprises the background of the project, the reasons for taking it, problems addressed by the project and expected outcomes. A good report starts with an introduction to the title of the project. The necessary background information is provided to establish context of the project. The motivation and significance of the project should be highlighted. A crisp problem statement is followed by scope of the project along with any limitations or exclusions. The specific objectives to be achieved should be stated. A roadmap or organization of report concludes the chapter.

1.1 Overview

WEJHA is a smart stadium navigation mobile application developed to assist football fans and event attendees in reaching their seats efficiently inside large stadiums. The application focuses on solving navigation and crowd-management challenges commonly experienced during large events.

The system allows users to view their match tickets, identify the correct gate and seating area, and receive the best path to their assigned seat through an interactive stadium map. The project is initially targeted toward Alinma Stadium (King Abdullah Sports City) in Jeddah, Saudi Arabia.

The application uses digital mapping and routing technologies to provide navigation services inside the stadium environment. It also includes crowd monitoring features that help reduce congestion and improve traffic flow inside stadium facilities.

The system consists of two main user types:

1. Regular Users (Fans)
2. Administrators (Stadium Management)

The application is developed using Flutter and Supabase technologies to support Android and iOS platforms

1.2 Project Motivation

Large football matches and entertainment events often suffer from severe crowd congestion inside stadiums. Visitors usually face difficulties locating their assigned gate, section, and seat, especially during high-attendance events. These problems lead to delays, confusion, stress, and unsafe crowd conditions.

Most current ticketing systems focus only on ticket purchasing and do not provide indoor navigation or intelligent guidance after entering the stadium. As a result, many attendees waste time searching for the correct entrances and seating locations.

The motivation behind WEJHA is to improve the fan experience by introducing a smart navigation solution specifically designed for stadium environments. The project combines indoor mapping, routing algorithms, and crowd-awareness concepts to create a safer and more organized experience for users.

Additionally, the project supports stadium management by providing real-time crowd indicators and notification systems that help administrators manage gates and corridors more effectively.

1.3 Problem Statement

During major sports events, stadium visitors encounter several challenges related to navigation and crowd movement. Common issues include:

- Difficulty identifying the correct stadium gate.
- Problems locating sections and seat numbers.
- Heavy congestion at certain entrances and corridors.
- Delays in reaching seats before the match begins.
- Lack of guidance inside large stadiums.

Existing ticketing applications do not provide effective indoor navigation or crowd-based routing solutions. Consequently, visitors experience confusion and overcrowding, which negatively impacts safety and user satisfaction.

The WEJHA application addresses these issues by providing an interactive navigation system that guides users from the entrance gate to their seats using optimized paths and real-time crowd information

1.4 Project Aim and Objectives

1.4.1 Project Aim

The primary aim of WEJHA is to develop a smart mobile application that enhances stadium navigation and improves crowd management during large events.

1.4.2 Project Objectives

The project objectives include:

- Designing an interactive indoor stadium map.
- Providing navigation from gates to seats.
- Reducing crowd congestion inside stadiums.

- Supporting dynamic route updates based on crowd levels.
- Displaying ticket information digitally.
- Providing administrators with monitoring tools.
- Sending real-time notifications to users.
- Supporting both Android and iOS platforms.
- Improving overall visitor safety and experience.

The project achieves these goals through mobile technologies, routing systems, real-time databases, and crowd analysis mechanisms.

1.5 Related Existing Systems

The following systems represent the current state of the market that WEJHA seeks to improve upon. The comparison table below highlights key feature gaps:

Feature	WEJHA	Webook	Ticketmaster	Google Maps
Indoor Navigation	✓	✗	✗	Partial
Seat-Level Routing	✓	✗	✗	✗
Real-Time Crowd Monitoring	✓	✗	✗	✗
AI Navigation Chatbot	✓	✗	✗	✗
Admin Dashboard	✓	✗	Limited	✗
Push Notifications	✓	Email only	Limited	✗
Arabic RTL Support	✓	✓	Partial	✓

Table 1.1 : Related Existing Systems

CHAPTER 2: PLANNING PHASE

2.1 Scope of the project

The scope of WEJHA includes the development of a mobile application that provides navigation services inside stadiums.

2.1.1 Included Features

- User registration and login.
- Ticket viewing.
- Indoor stadium maps.
- Gate-to-seat navigation.
- Crowd monitoring.
- Notifications system.
- Admin dashboard.
- Dynamic routing support.
- ChatBot

2.1.2 Excluded Features

- Online ticket purchasing or payment processing.
- Integration with external ticketing APIs
- Outdoor GPS navigation to the stadium entrance.
- Web application or browser-based interface.
- AI-based predictive crowd forecasting.

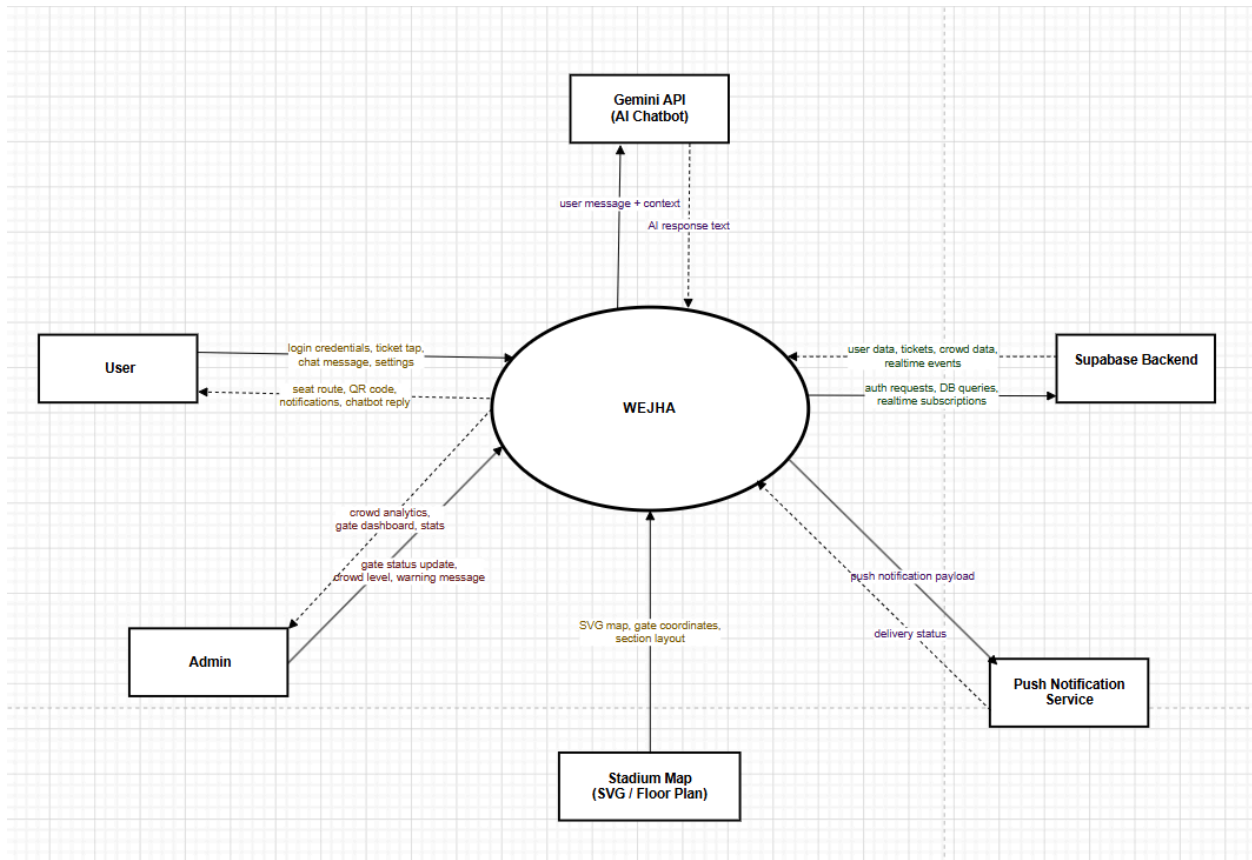


Figure 2.1 : Context Diagram

2.2 Project risks and Product risks

Risk	Description	Severity	Mitigation
Map Complexity	Stadium canvas rendering at pixel accuracy is time-intensive	High	Iterative prototyping; fixed coordinate system
Routing Accuracy	Waypoint path errors could mislead users in safety-critical	High	Comprehensive path testing for each gate/section combination

	scenarios		
Supabase Real-Time Latency	Crowd data delays may cause stale routing recommendations	Medium	Subscription health monitoring; graceful fallback to last-known data
Gemini API Availability	AI chatbot unavailable if API quota exceeded or key invalid	Medium	Multi-model fallback (flash-2.0-lite → flash-1.5); graceful error UI
Platform Fragmentation	UI rendering differences between Android and iOS	Low	Flutter widget abstraction; test on both platforms
Schedule Risk	Academic deadlines limiting full-feature completion	Medium	MoSCoW prioritization; core navigation delivered first

Table 2.1 : Project risks and Product risks

2.3 Feasibility study

2.3.1 Technical Feasibility

The project is technically feasible because Flutter supports cross-platform mobile development and Supabase provides scalable backend services including authentication, database management, and realtime updates.

2.3.2 Schedule Feasibility

The project can be completed within the academic timeline through proper task distribution and agile development practices.

2.3.3 Operational Feasibility

The application is practical for stadium visitors and management teams and can improve operational efficiency during even

2.4 Project Schedule

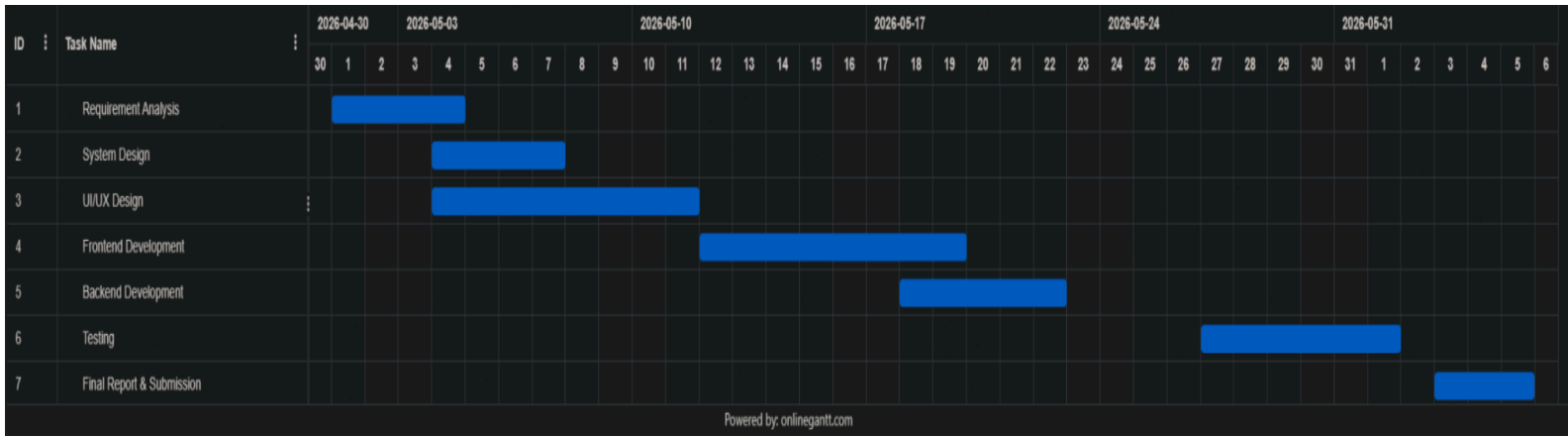


Figure 2.1: Gantt Chart

2.5 Project Software and Hardware Requirements

2.5.1 Software Requirements

- Flutter
- Dart Programming Language
- Supabase
- PostgreSQL
- Visual Studio Code
- Figma
- GitHub

2.5.2 Hardware Requirements

- Development laptops/computers

- Android testing devices
- iPhone testing devices
- Internet connection

CHAPTER 3: REQUIREMENT ENGINEERING AND ANALYSIS

3.1 Used Techniques For Requirements Collection

The following techniques were used to collect project requirements:

- Interviews with football fans.
- Observation of crowd movement during matches.
- Analysis of existing ticketing applications.
- Brainstorming sessions.
- Online surveys.

3.2 Functional Requirement & Modelling

Functional Requirement (FR) is a description of the service that the software must offer. It describes a software system or its component. A function is nothing but inputs to the software system, its behavior, and outputs.

3.2.1 List of System Functions (Features)

ID	Actor	Feature
F1	User	Register account

F2	User/Admin	Login/logout
F3	User	View tickets
F4	User	View QR code
F5	User	Navigate to seat
F6	User	AI Chat Bot
F7	User	Receive notifications
F8	Admin	Manage gates
F9	Admin	View crowd in gates
F10	Admin	Real-time dashboard
F11	Admin	Send warning message

Table 3.1 : Features

3.2.2 Use Case Diagram

The use case diagram captures the interactions between system actors and the main functionalities of Wejha . The primary actors are: Admin , User and Gemini API.

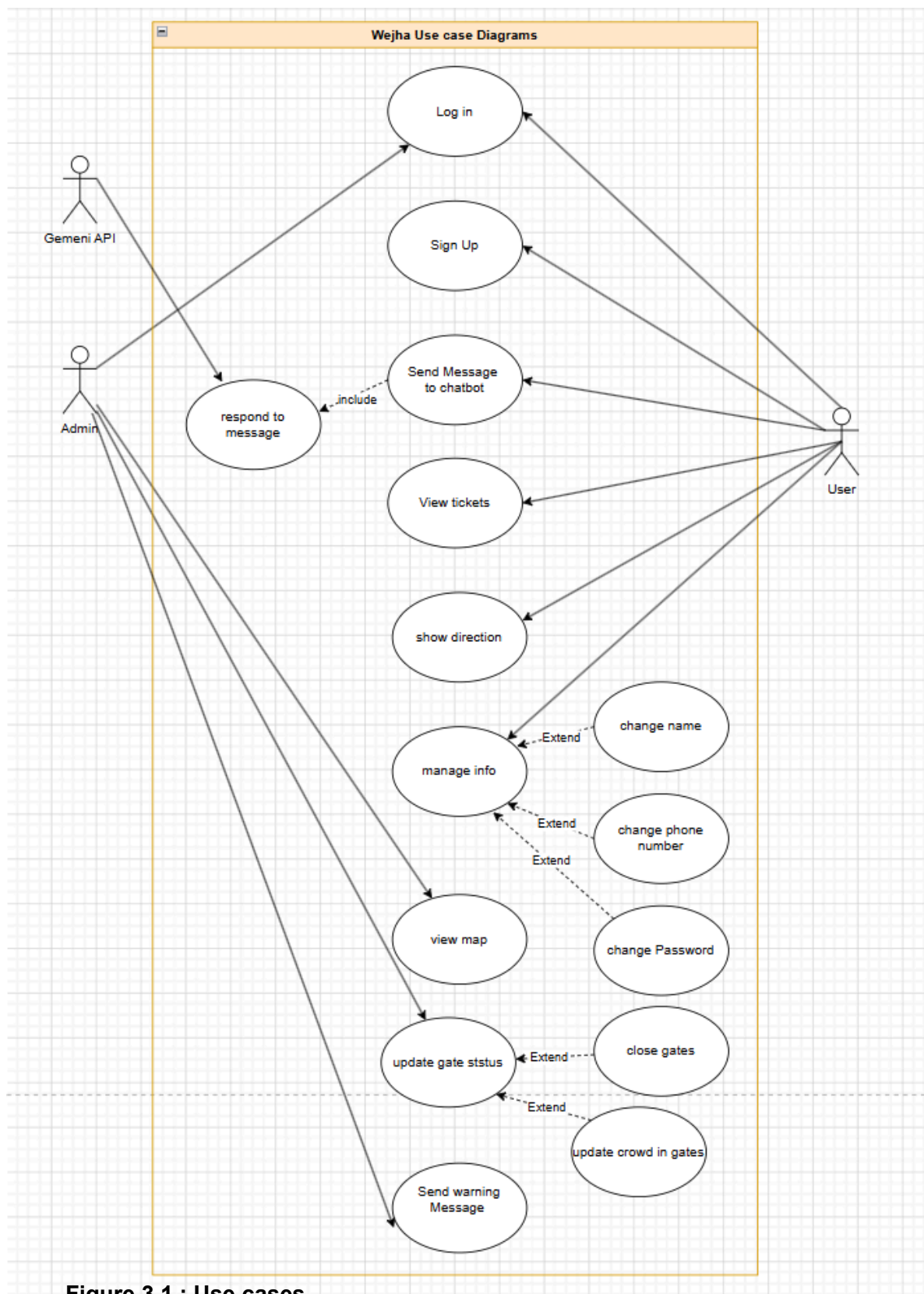


Figure 3.1 : Use cases

3.2.3 Use Cases: Description & Details

Each major use case is documented below with its actor(s), description, main flow, and alternative flows.

Use Case 1: Sign Up

Use Case ID	UC-01
Actor	User
Description	Allows new users to create an account in the Wejha application by providing personal information such as full name, Email , password, and phone number.
Pre-condition	User has installed the Wejha application and is not currently logged in.
Main Flow	1. User opens the Wejha application. 2. User selects "Create Account". 3. System displays the registration form. 4. User enters full name, email, phone number, and password. 5. User confirms the password. 6. User submits the registration form. 7. System validates all entered data. 8. System creates a new account in Supabase Authentication. 9. User profile is created in the database. 10. System displays a successful registration message. 11. User is redirected to the Home Screen.
Alternative Flow	Invalid or missing information → System displays validation error. Email already exists → System suggests login. System error → Registration fails and user is asked to try again later.
Post-condition	A new user account is created successfully and can access the system.

Table 3.2 : Sign Up

Use Case 2: Login

Use Case ID	UC-02
Actor	Registered User, Admin
Description	Allows registered users and administrators to access the Wejha application using their email and password.
Pre-condition	User Already has a registered account in the system.
Main Flow	1. User opens the Wejha application. 2. User selects "Login". 3. System displays the login form. 4. User enters email and password. 5. User submits the login request. 6. System validates the credentials. 7. System authenticates the user through Supabase Authentication. 8. User session is created. 9. System redirects the user to the Home Screen or Admin Dashboard based on their role.
Alternative Flow	4a. User leaves required fields empty → System highlights missing fields. 6a. Invalid email or password → System displays an authentication error message. 7a. User account does not exist → System suggests creating a new account. 7b. Authentication service is unavailable → System displays an error message and asks the user to try again later.
Post-condition	User is successfully authenticated and granted access to the appropriate application features.

Table 3.3 : Login

Use Case 3: View Tickets

Use Case ID	UC-03
Actor	Registered User
Description	Allows users to view all tickets associated with their account.
Pre-condition	User is logged in and has one or more tickets assigned to their account.

Main Flow	1. User navigates to the "My Tickets" screen. 2. System retrieves ticket data from the database. 3. System displays the list of available tickets. 4. User views ticket information including match name, date, stadium, and status. 5. User selects a ticket to view more details.
Alternative Flow	2a. System fails to retrieve ticket data → Error message is displayed. 3a. User has no tickets → System displays an empty state message. 3b. Internet connection is unavailable → System asks the user to try again later.
Post-condition	User successfully views their available tickets.

Table 3.4 : View Tickets

Use Case 4: View QR code

Use Case ID	UC-04
Actor	Registered User
Description	Allows users to view the QR code associated with their ticket for stadium entry and verification.
Pre-condition	User is logged in and has at least one valid ticket.
Main Flow	1. User navigates to the My Tickets screen. 2. User selects a ticket. 3. User chooses "View QR Code". 4. System retrieves the QR code. 5. System displays the QR code on the screen.
Alternative Flow	4a. QR code cannot be retrieved → System displays an error message. 4b. Ticket is invalid or expired → System notifies the user.
Post-condition	User successfully views the QR code for the selected ticket.

Table 3.5 : View QR code

Use Case 5: Manage gates

Use Case ID	UC-05
Actor	Admin
Description	Allows administrators to monitor and manage stadium gates by updating their status and crowd levels.
Pre-condition	Admin is logged into the system and has access to the Admin Dashboard.
Main Flow	1. Admin navigates to the Gate Management screen. 2. System displays all stadium gates and their current status. 3. Admin selects a gate. 4. Admin updates the gate status (Open/Closed). 5. Admin updates the crowd level if necessary. 6. System saves the changes. 7. System updates the gate information in real time.
Alternative Flow	4a. Invalid gate information is entered → System displays a validation error. 6a. System fails to save changes → System displays an error message. 7a. Network connection is unavailable → Changes cannot be synchronized until connection is restored.
Post-condition	Gate status and crowd information are successfully updated and reflected in the system.

Table 3.6 : Manage Gates

Use Case 6: Real-time dashboard

Use Case ID	UC-06
Actor	Admin
Description	Allows administrators to monitor live stadium information, including crowd levels, gate status, active users, and ticket statistics.
Pre-condition	Admin is logged into the system and has access to the Admin Dashboard.

Main Flow	1. Admin opens the Dashboard screen. 2. System retrieves real-time data from Supabase. 3. System displays crowd levels by gate and section. 4. System displays gate statuses (Open/Closed). 5. System displays active users and ticket statistics. 6. Dashboard updates automatically when data changes.
Alternative Flow	2a. Real-time data cannot be retrieved → System displays an error message. 6a. Connection is lost → Dashboard shows the last available data until reconnection.
Post-condition	Admin successfully views the latest stadium and crowd information.

Table 3.7 : Real-time dashboard

Use Case 7: Send warning message

Use Case ID	UC-07
Actor	Admin
Description	Allows administrators to send warning or alert messages to users regarding crowd conditions, gate status, or stadium announcements.
Pre-condition	Admin is logged into the system and has access to the Notifications Management screen.
Main Flow	1. Admin opens the Notifications screen. 2. Admin enters a warning message. 3. Admin selects the target users or audience. 4. Admin sends the notification. 5. System delivers the message to users. 6. Users receive the notification in the app.
Alternative Flow	4a. Message content is empty → System displays a validation error. 5a. Notification delivery fails → System displays an error message.
Post-condition	Warning message is successfully sent and delivered to users.

Table 3.8 : Send Warning message

3.4 Nonfunctional Requirements: Quality & Constraints

Attribute	Metric Requirement Specification	Implementation Implementation Strategy
Performance	Database stream changes must update client maps within 1.5 seconds.	Uses low-overhead Supabase PostgreSQL WAL replication streams.
Security	Users must be restricted from reading other attendees' ticketing profiles.	Enabled strict PostgreSQL Row Level Security (RLS) checked via user tokens.
Availability	Core structural navigation and local map viewing must remain online 24/7.	Local asset caching of maps and static coordinates via SharedPreferences.
Localization	Total Right-to-Left (RTL) interface alignment with Cairo typographic weights.	Centralized system using GoogleFonts.cairo and clean RTL padding.

Table 3.1 : Nonfunctional Requirements

CHAPTER 4: ARCHITECTURE & DESIGN

This chapter describes the software architecture and detailed design of the WEJHA application. It presents the overall architectural structure, the internal interactions of key use cases through sequence diagrams, the class diagram, the database organization, and the user interface prototypes.

4.1 Software Architecture

WEJHA adopts a cloud-backed mobile architecture built around three interconnected tiers: the user-facing Flutter application, the Supabase Backend-as-a-Service (BaaS) layer, and the Google Gemini AI chatbot service. Figure 4.1 illustrates a simplified overview of how these components interact.

WEJHA App (User Layer): The primary client is a Flutter mobile application targeting both Android and iOS platforms. It exposes four core modules to regular users: Ticket QR Code viewing, an SVG-based interactive Stadium Map, Path Navigation, and a Gemini-powered AI Chatbot. The app communicates with the Supabase backend over HTTPS for authentication and data requests, and maintains a persistent WebSocket connection to receive real-time updates such as gate status changes and crowd level shifts.

Supabase Backend (BaaS Layer): Supabase serves as the central backend, integrating three services within a single managed platform: Auth for user authentication and session control, a PostgreSQL Database secured with Row-Level Security (RLS) policies, and a Realtime engine for broadcasting live data over WebSocket. All authentication flows, persistent data storage, and real-time state propagation pass through this layer. The backend also acts as the intermediary between the user app and the Gemini AI service, forwarding chatbot queries over HTTPS and relaying responses back to the client.

Admin App (Admin Layer): A dedicated Flutter application provides stadium administrators with a Management dashboard, a Notifications panel for broadcasting alerts, and an Analytics view for monitoring crowd conditions. The Admin App exchanges data and state updates with Supabase over both HTTPS and WebSocket connections, enabling administrators to manage gate statuses and respond to crowd events in real time.

Gemini AI (Chatbot Service): Google Gemini AI serves as the conversational intelligence behind the in-app chatbot. The Supabase backend forwards user queries to the Gemini API over HTTPS and returns responses to the app. The chatbot is context-aware, with access to the user's ticket details and live crowd information,

enabling it to provide relevant navigation guidance. The chatbot communication flow is visually distinguished in the architecture diagram using dashed lines.

Communication Protocols: The architecture relies on three distinct communication patterns: standard HTTPS request/response for data exchange and AI queries, WebSocket connections for real-time bidirectional updates between the app and Supabase, and a dedicated HTTPS-based chatbot flow connecting the user app, Supabase, and Gemini AI.

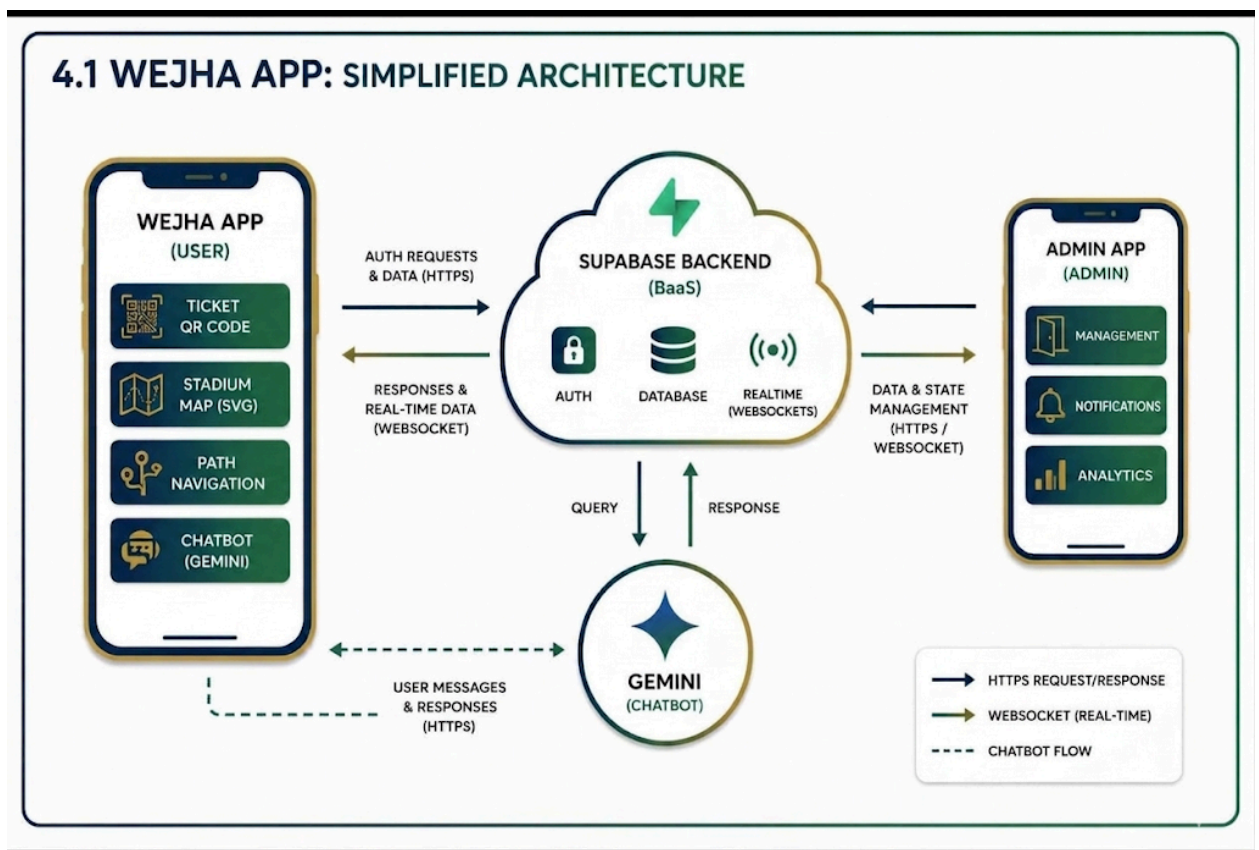


Figure 4.1 : Simplified Architecture

4.1 Software Detailed Design

4.1.1 Use Cases Internal Interactions as Sequence Diagrams

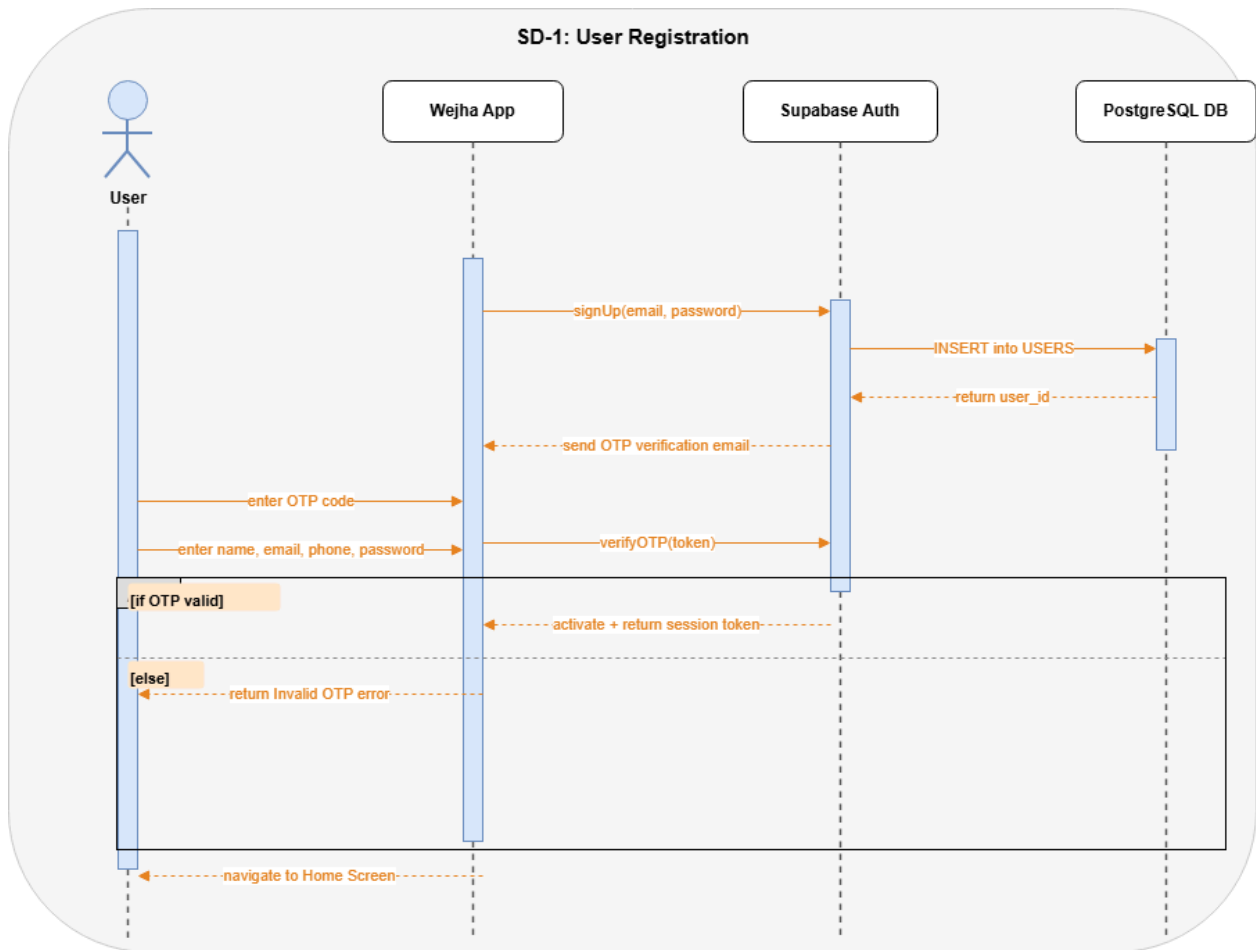


Figure 4.1.2 :Sequence diagram of User Registration

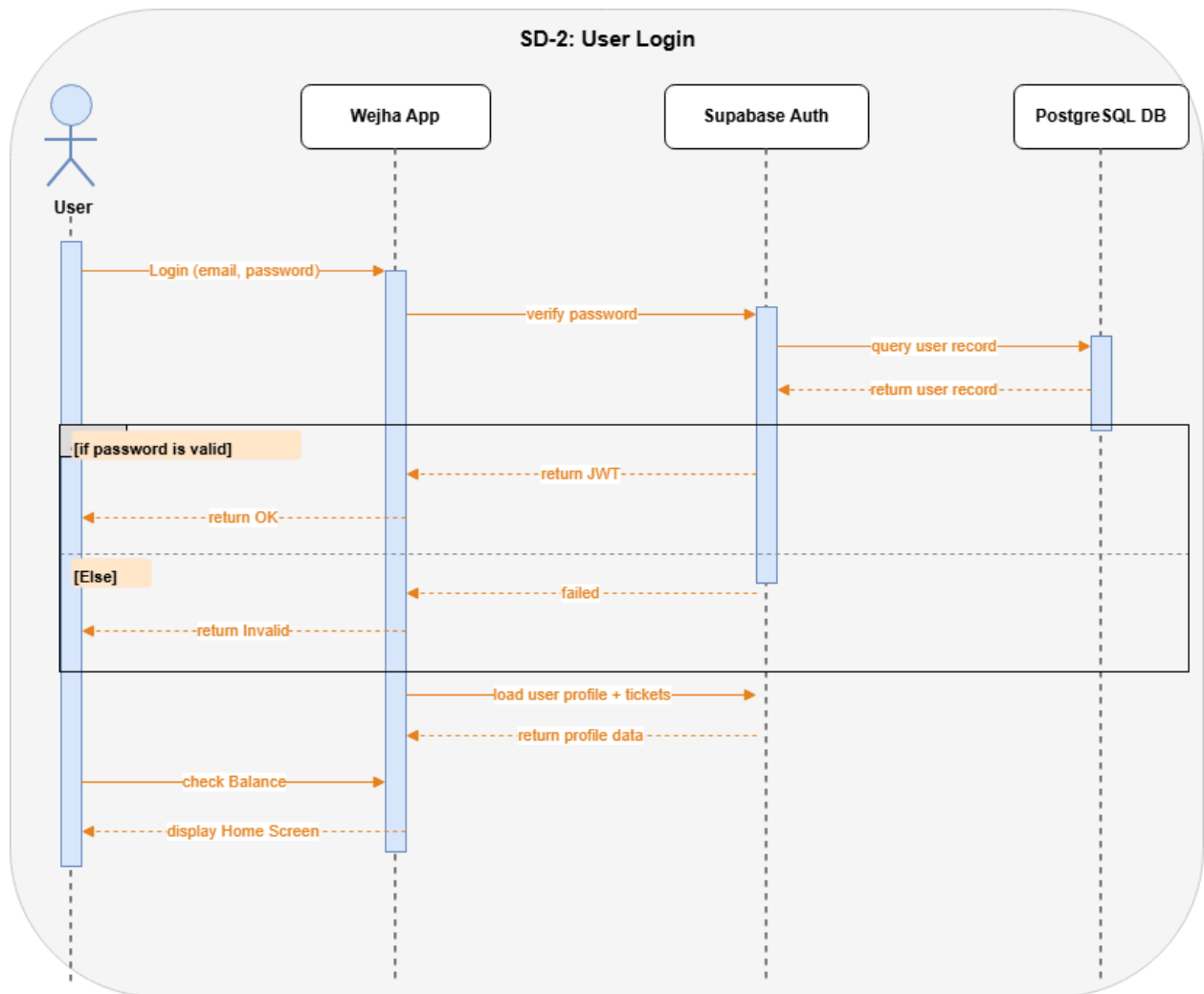


Figure 4.1.3 : Sequence diagram of User Login

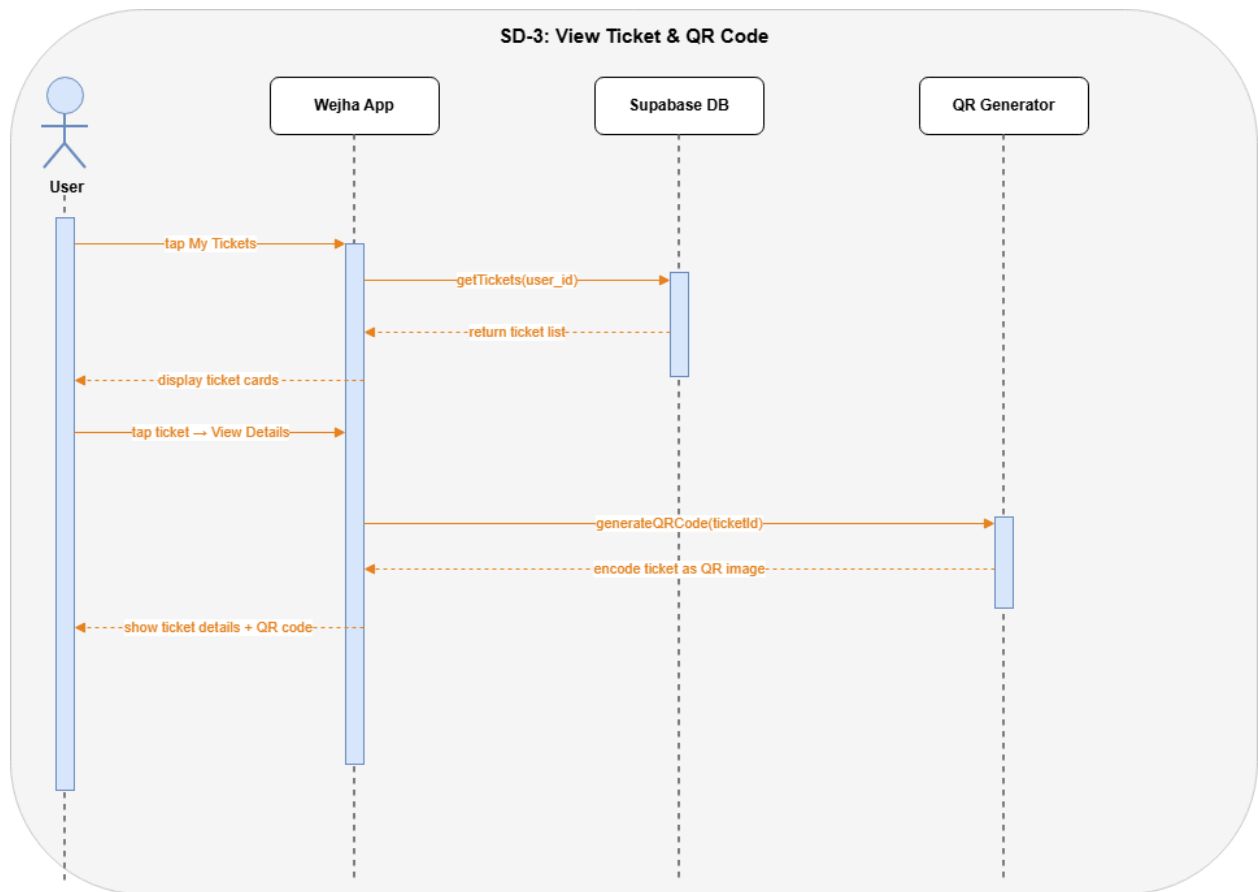


Figure 4.1.4 : Sequence diagram of View Ticket & QR Code

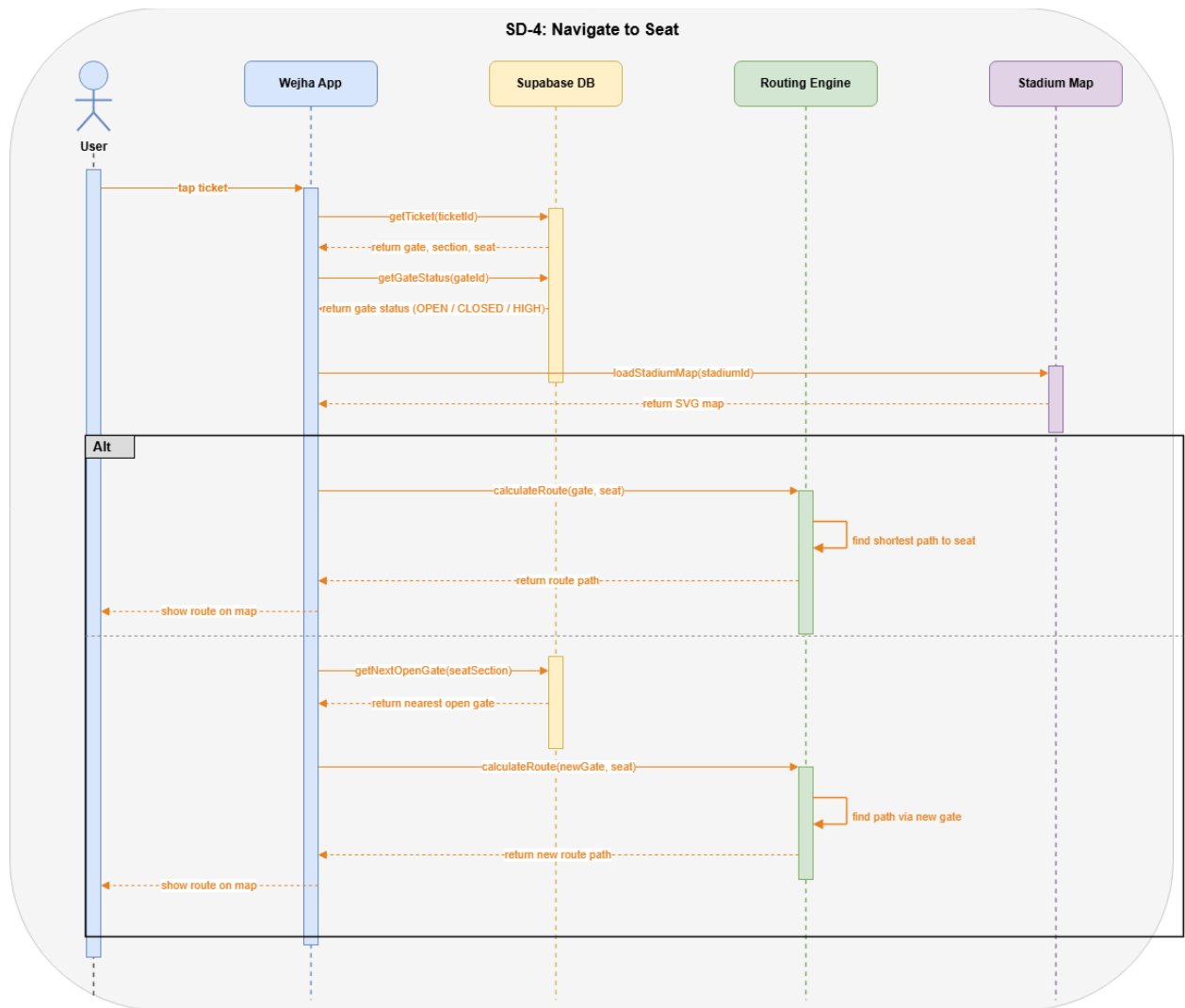


Figure 4.1.5 : Sequence diagram of Navigate to Seat

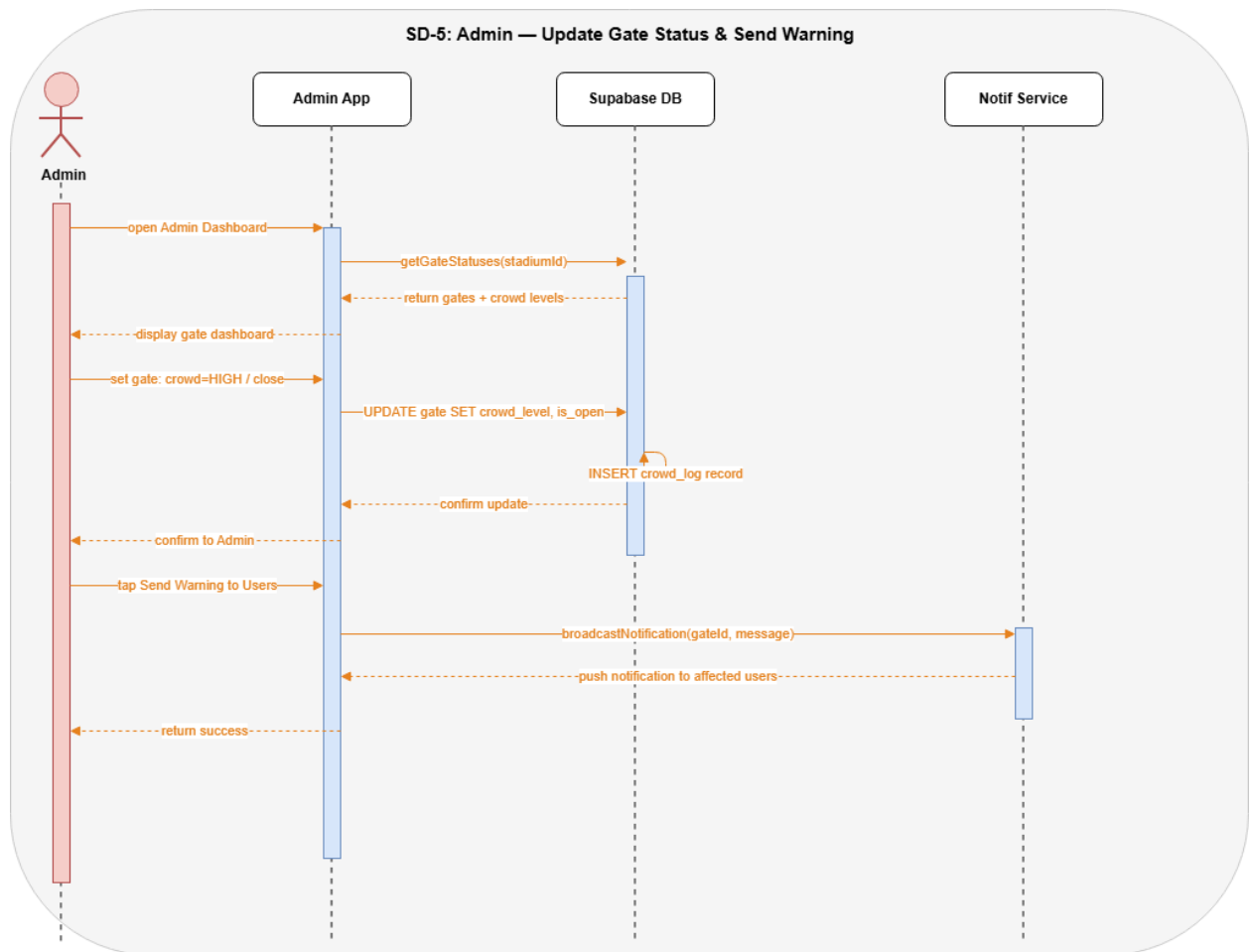


Figure 4.1.6 : Sequence diagram of Update Gate Status & Send Warning

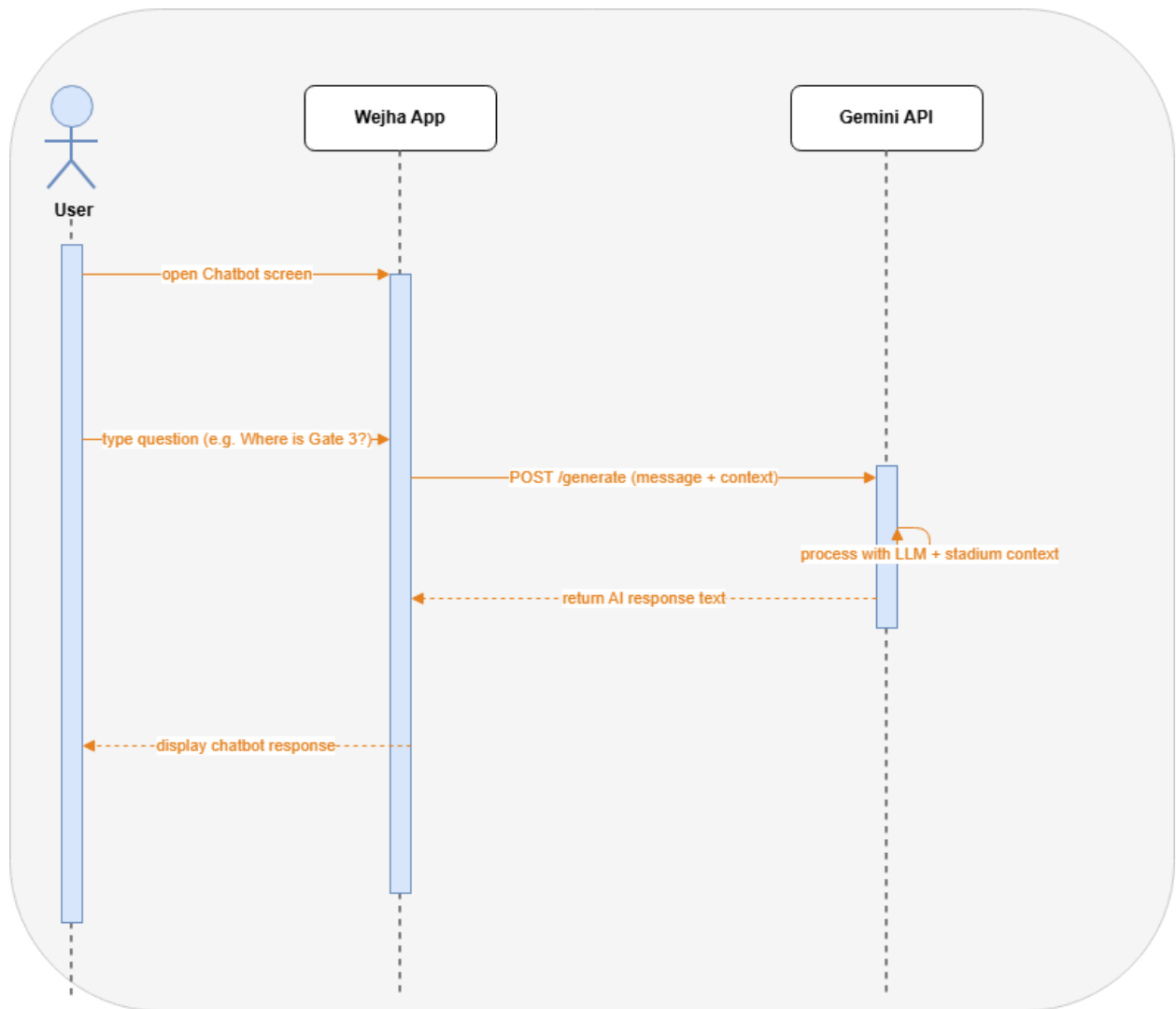


Figure 4.1.7 : Chatbot interaction

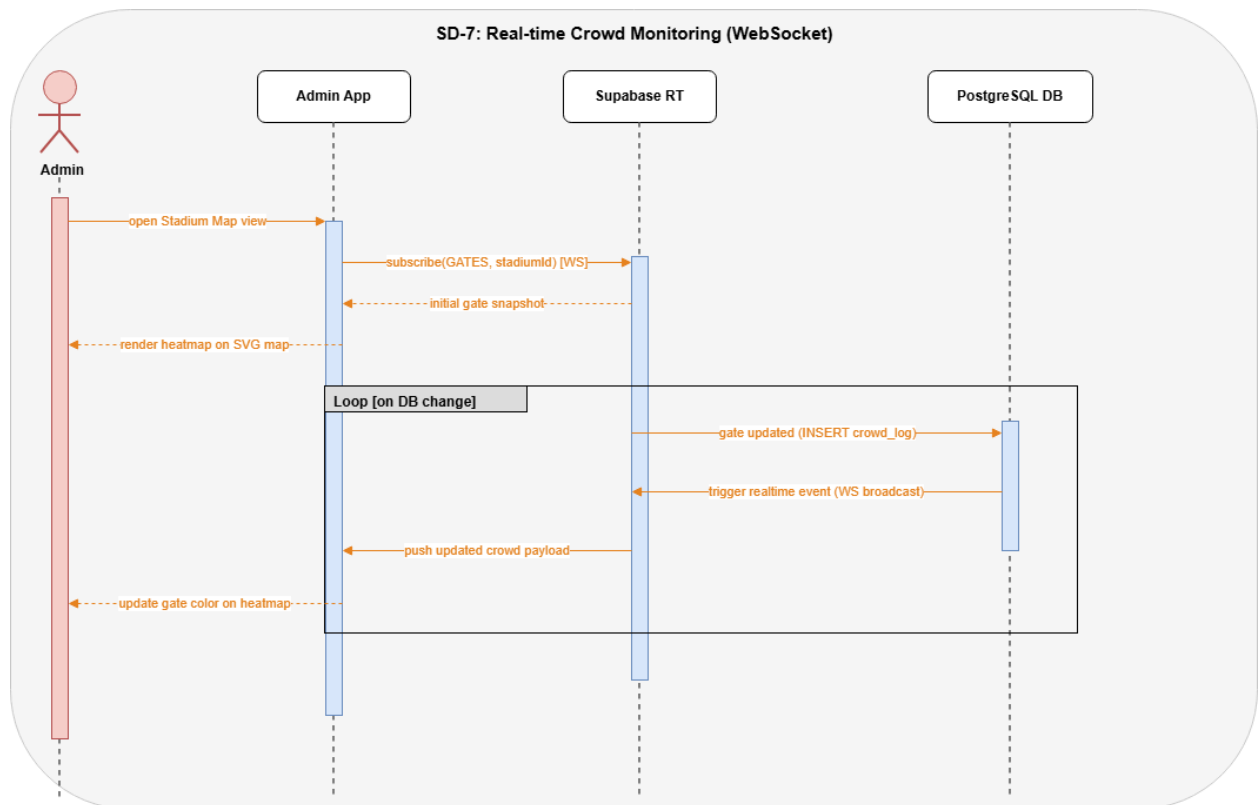


Figure 41.8 : Sequence diagram of Real-time Crowded Monitoring

4.1.2 Class Diagram

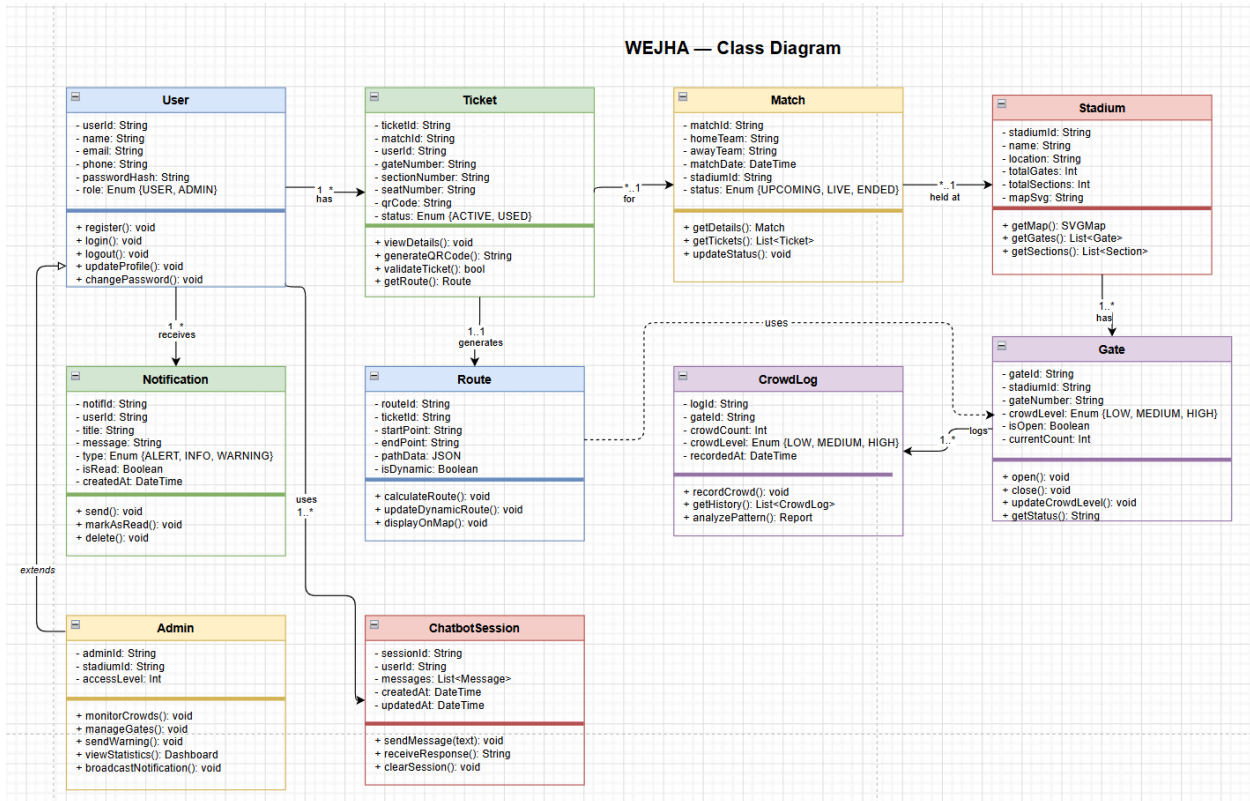


Figure 4.1.9 : Class Diagram

4.1.3 Data Storage Organization

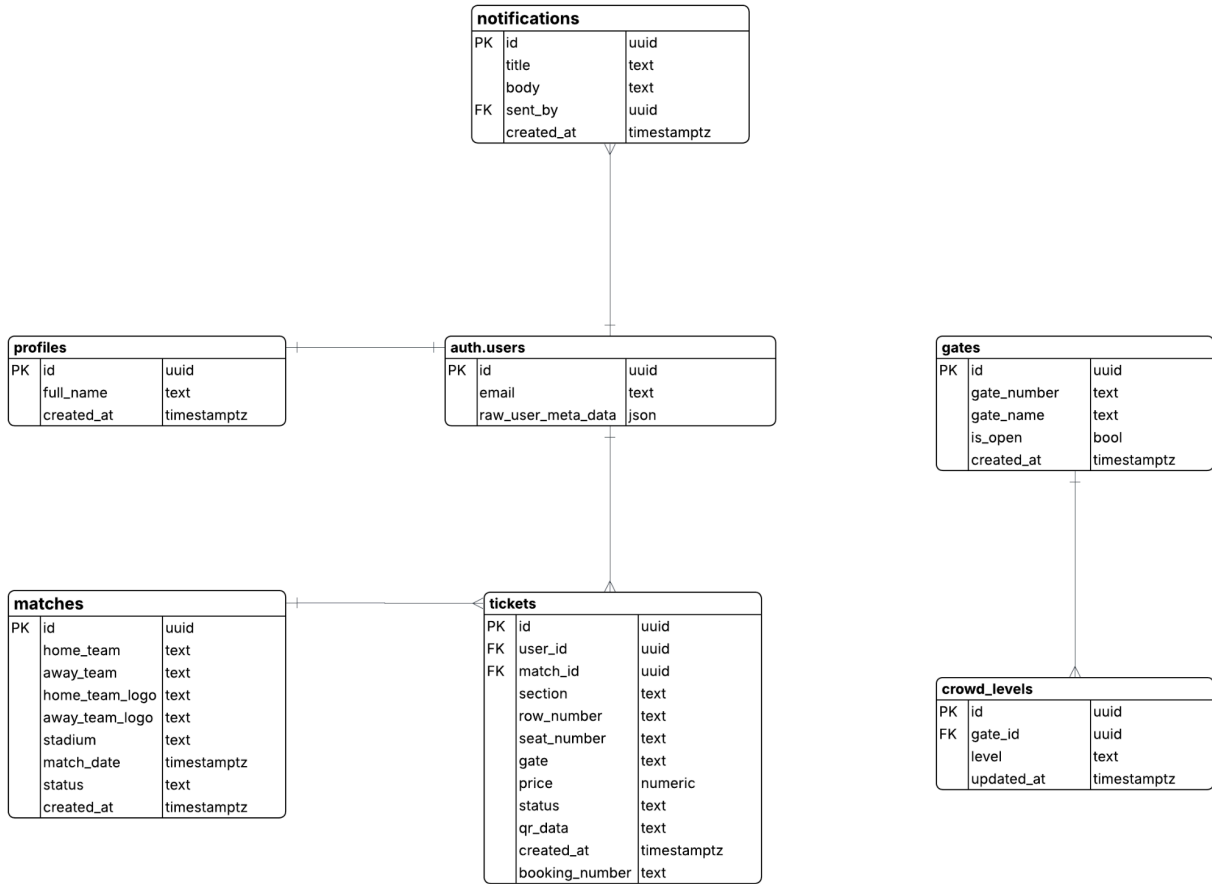


Figure 4.1.10 : ERD Data Organization

4.2 User Interface Prototyping (ONLY FOR PROJECT 1)

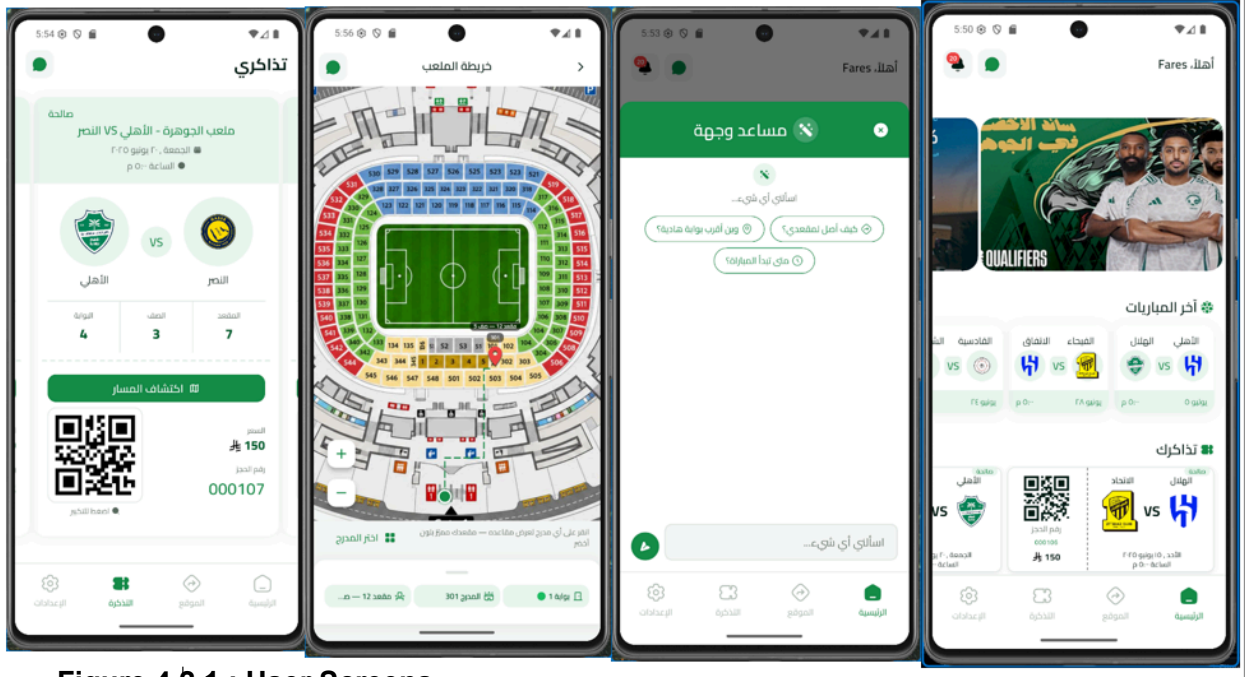


Figure 4.2.1 : User Screens



Figure 4.2.2 : Admin Screens

Screen	Description
Home	The main entry point of the app, showing upcoming matches, recent ticket summaries with QR codes, and team listings.
Chatbot	A conversational assistant powered by Google Gemini AI that accepts natural language queries and provides personalized navigation guidance based on the user's ticket and real-time crowd data.
User Map	Renders an SVG-based aerial view of Alinma Stadium with color-coded seating sections and zoom controls, allowing the user to visually locate their assigned gate and section.
Ticket View	Displays match details including team names (Al-Ahli vs. Al-Nasr), team logos, match statistics, seat number, ticket price, and a scannable QR code for stadium entry.

Table 4.2.1 : User Screens

Screen	Description
Admin Dashboard	A centralized control panel summarizing key metrics including active specialists, open events, total subscribers, and crowd alerts, alongside quick-access sections for gate status and match ticket counts.
Gate Status & Crowd	Lists all stadium gates with their real-time open/closed toggle status and crowd level indicators (high, medium, low) per gate, with a broadcast button to send crowd warnings to users.
Stadium Map View	Displays a heat-map overlay on the stadium layout with color indicators — red for congested zones, orange for

	moderate, and green for clear — giving administrators a spatial overview of crowd distribution.
--	---

Table 4.2.2 : Admin Screen

CHAPTER 5: TESTING PLAN

5.1 Tools

Tool / Framework	Purpose
Flutter	Cross-platform mobile UI framework
Dart	Programming language
Supabase	Backend: auth, database, real-time
Flutter Riverpod	State management & dependency injection
GoRouter	Declarative navigation & route guards
Google Gemini AI	AI chatbot natural language assistant
fl_chart	Admin dashboard charts & graphs
qr_flutter	QR code rendering for tickets
Visual Studio Code	Primary IDE
Figma	UI/UX prototyping
GitHub	Version control & collaboration

Table 5.1 : Tools and frameworks

5.2 Unit test

Testcase ID	Requirement	descriptions	Preconditions	Steps	Expected result	Actual result	Status
UT-01	FR6 - User Authentication	Widget test: LoginScreen renders all required UI elements without errors	App is launched; user is unauthenticated	1. Pump LoginScreen widget 2. Verify widget tree	Email field, password field, login button, and app logo are visible	All UI elements rendered correctly	Pass
UT-02	FR6 - Input Validation	Verify error appears when email field is empty on login	LoginScreen is rendered	1. Leave email empty 2. Tap login button	Error message 'Email cannot be empty' shown below email field	Validation message displayed	Pass
UT-03	FR6 - Input Validation	Verify error appears for invalid email format	LoginScreen is rendered	1. Enter 'not-an-email' 2. Tap login button	Error message 'Enter a valid email' appears	Format validation triggered	Pass
UT-04	FR6 - Input Validation	Verify error for password shorter than 6	LoginScreen is rendered	1. Enter '123' in password 2. Tap login	Error message 'Password must be at least 6 characters' appears	Length validation works	Pass

		characters		button	least 6 characters'		
UT-05	FR6 - Input Validation	Verify error when password field is empty	LoginScreen is rendered	1. Leave password empty 2. Tap login button	Error message 'Password cannot be empty'	Empty field validation works	Pass
UT-06	FR6 - Auth Service	Successful login navigates user to HomeScreen	Valid user exists in Supabase	1. Enter valid email + password 2. Tap login	User authenticated; app navigates to HomeScreen	Auth success + navigation	Pass
UT-07	FR6 - Auth Service	Wrong credentials shows error message	Mock auth throws exception	1. Enter wrong email/password 2. Tap login	Error message 'Invalid credentials' displayed on screen	Exception caught and shown	Pass
UT-08	FR2 - Live Dashboard	Dashboard refresh timer triggers every 10 seconds	Dashboard is active	1. Advance fake timer by 10 s 2. Verify rebuild	Widget rebuilds and calls data fetch after 10 s interval	Timer triggers correctly	Fail
UT-09	FR2 - Live Dashboard	Dashboard displays crowd density level correctly	DensityReport mock returns level 'High'	1. Inject mock DensityReport 2. Inspect displayed text	Text 'High' visible in crowd status widget	Density level displayed	Pass
UT-10	FR2 - Live Dashboard	Dashboard	DensityReport	1. Inject mock	Text 'High'	Density level displayed	Pass

		displays crowd density level correctly	mock returns level 'High'	DensityReport 2. Inspect displayed text	visible in crowd status widget		
UT-11	FR7 - Notification Content	Emergency notification contain alert type	Alert triggered	1. Capture notification payload 2. Inspect fields	Payload includes 'alertType'	Fields present	Pass
UT-12	FR4 - Ticketing API	retrieveTicketInfo() returns Ticket for valid attendeeID	Mock Ticketing API returns data	1. Call retrieveTicketInfo('ATT-001') 2. Inspect result	Returns Ticket with ticketID, seatNumber, and eventDate	Ticket returned	Pass
UT-13	FR5 - Analytics / Heatmap	Heatmap Widget renders with correct color zones	Mock density data provided	1. Inject density data 2. Pump Heatmap Widget	Red zones for >80%, yellow 50-80%, green <50%	Colors correct	Pass
UT-14	FR12 – Facility Info	FacilityService.getAmenities() returns list of facility items	Mock amenity data provided	1. Call getAmenities() 2. Inspect list	Returns list containing at least: restrooms, food stalls, first aid	Feature not activated	Fail

Table 5.2 : Unit tests

Testcase ID	Requirement	descriptions	Preconditions	Steps	Expected result	Actual result	
ST-01	FR6 – End-to-End Login	Full login flow: valid credentials → authenticated session	Supabase running; user account exists	1. Open app 2. Enter valid email + password 3. Tap Login	User authenticated; redirected to HomeScreen; session token stored	Full flow works	Pass
ST-02	FR6 – End-to-End Login	Full login flow: wrong credentials → error displayed	App running	1. Enter wrong credentials 2. Tap Login	Error snack bar shown; user stays on LoginScreen	Error flow correct	Pass
ST-03	FR1 – Real-Time Crowd Monitor	Admin views live crowd density from Supabase sensor data	Admin logged in; sensor data streaming	1. Admin navigates to Dashboard 2. Waits 10 s	Dashboard shows updated density levels per zone within 10 s	Live data displayed	Pass
ST-04	FR2 – Dashboard Latency	Dashboard data refreshes within max 2-second latency	Network with <100 ms ping	1. Trigger dashboard load 2. Measure time to display	Data appears within 2 seconds	The App is not connected to server	Fail
ST-05	FR3 – Dynamic Routing	System reroutes attendee when the assigned gate becomes unavailable	Gate 4 is closed	1. Attendee seat is assigned to Gate 4 route. 2. Close Gate 4. 3. Request	Navigation is automatically updated to the nearest available gate (e.g., Gate 3)	Rerouting triggered to Gate 3	Pass

				navigation to seat.	and a new route is displayed.		
ST-06	FR4 – Ticketing API Integration	Attendee retrieves ticket info after login via ticketing API	User logged in; ticketing API connected	1. Navigate to Ticket screen 2. Tap 'View Ticket'	Ticket details (seat, gate, match) shown from external API	API integration works	Pass
ST-07	FR7 – Emergency Alert End-to-End	Emergency alert sent by admin reaches all devices within 5 s	Admin + 1 test devices connected	1. Admin triggers emergency alert 2. Measure time on device	The devices receive alert within 5 seconds	Alert delivered	Pass
ST-08	FR8 – Navigation Full Flow	Attendee navigates from entrance to seat using interactive map	User logged in; ticket has seat A12	1. Open App 2. Tap 'View Ticket' 3. Select 'Ticket A12'	Turn-by-turn directions displayed	Navigation complete	Pass
ST-09	NFR1 – Concurrent Users	System handles 10,000 simultaneous connections without degradation	Load test environment ready	1. Simulate 10,000 concurrent logins 2. Monitor response time	Response time stays ≤ 2 s for all requests	System could not handle 10,000 concurrent users; significant performance degradation and request timeouts were observed.	Fail
ST-10	NFR4 – Uptime	System maintains 99.9% uptime during	Staging environment with monitoring	1. Run 72-hour uptime test 2.	Total downtime ≤ 4.32 minutes over 72	-	-

		simulated event		Measure downtime	hours		
ST-11	NFR3 – Data Update Frequency	Admin dashboard data updates every 10 seconds	Dashboar d active for 2 minutes	1. Monitor dashboard 2. Count data updates in 2 min	At least 12 updates in 2 minutes	-	-
ST-12	FR10 – Language Switching	Full UI switches to English without restart	App running in Arabic	1. Open Settings. 2. Select English. 3. Navigate all screens.	All screens display English labels; layout is updated appropriately (LTR).	-	-
ST-13	NFR5 – Chatbot Response Time	The system chatbot shall respond to user requests within a maximum of 3 seconds to ensure a smooth and efficient user experience.	Chatbot service running	1. Submit a user query to the chatbot 2. Measure response time"	Response is received within 3 seconds	Chatbot response time exceeded the required limit and took approximately 7 seconds to respond	Fail

Table 5.3 : System tests

Test Case ID	Requirement	Description	Preconditions	Steps	Expected Result	Actual Result	Status
UAT-01	FR6 – Attendee	Attendee	Registere	1. Open	User	Login	Pass

	Login	can log in with event ticket credentials	d attendee account	Wejha app 2. Enter credentials 3. Tap Login	sees HomeScreen with their seat and gate info	accepted by user	
UAT-02	FR8 – Seat Navigation	Attendee finds their seat using the in-app map	User logged in; ticket assigned	1. Open Wejha App 2. Sign in 3. Tap 'View Ticket' 4. Tap 'My seat'	Step-by-step directions lead to correct seat in ≤3 min	Attendee reached seat	Pass
UAT-03	FR3 – Crowd Avoidance	App warns attendee about crowded area and suggests alternate path	Dense area on default route	1. Start navigation 2. Observe rerouting	In-app message : 'This path is crowded. Try Gate 3 instead.'	Warning helpful	Pass
UAT-04	FR7 – Receive Emergency Alert	Attendee receives emergency notification on device	Emergency triggered by admin	1. Admin triggers alert 2. Wait on attendee device	Push notification appears with clear instructions	Alert received	Pass
UAT-05	FR4 – View Ticket	Attendee views full ticket details and QR code	Ticket in ticketing API	1. Open Ticket screen	Ticket shows match, seat, gate, QR code clearly	Ticket visible	Pass
UAT-06	FR12 – Find Facilities	Attendee finds restrooms and food stalls via app	Facility data loaded	1. Open Facilities 2. Select 'Restroom'	Map shows nearest restroom with distance and	The App did not show the nearest restroom	Fail

					directions		
UAT-07	FR10 – Arabic Language	Arabic-speaking attendee uses app fully in Arabic	App installed on Arabic device	1. Open app (Arabic locale) 2. Navigate all main screens	All text in Arabic; layout RTL; numbers correct	Arabic usable	Pass
UAT-08	FR10 – English Language	English-speaking attendee uses app in English	App installed	1. Set language to English 2. Navigate all screens	All text correct in English	The language does not change	Fail
UAT-09	FR3 – Route Clarity	Navigation directions are clear and easy to follow	Active navigation	1. Start navigation to seat 2. Attendee follows steps	All 5 test users reached destination without confusion	Directions clear	Pass
UAT-10	FR7 – Emergency Response Usability	Attendee understands emergency alert and takes correct action	Emergency alert sent	1. Trigger fire alert 2. Observe 10 test attendees	≥8 of 10 users followed correct evacuation route	Response adequate	Pass
UAT-11	FR1 – Admin Crowd Monitoring	Stadium admin monitors live crowd density from dashboard	Admin account; crowd data streaming	1. Admin logs in 2. Opens Dashboard 3. Views crowd data	Admin clearly sees which zones are crowded in real time	Dashboard useful	Pass
UAT-12	FR2 – Dashboard Comprehension	Admin understands	New admin account	1. Admin opens Dashboard	Admin identifies 3 most	Dashboard intuitive	Pass

		dashboard without training		rd for first time	crowded zones within 60 seconds		
UAT-13	FR7 – Admin Sends Emergency Alert	Admin sends emergency alert to all attendees	Admin on Dashboard	1. Tap 'Emergency Alert' 2. Write The Emergency Alert 3. Confirm	All connected devices receive alert within 5 seconds	Alert sent successfully	Pass
UAT-14	FR9 – Admin Performance Monitoring	Admin monitors system health during peak traffic	Event ongoing; high load	1. Open monitoring panel 2. View response times	Admin sees real-time error rate and latency graph	The app is not connected to server	Fail
UAT-15	NFR1 – System Stability During Event	System stays stable for full 90-minute match simulation	Full event simulation	1. Run 90-min load test 2. Simulate realistic usage patterns	Zero crashes; data remains consistent throughout	The app is not connected to server	Fail

Table 5.4 : User Acceptance tests

CONCLUSIONS

WEJHA presents a comprehensive software solution to a real and measurable problem experienced by millions of football fans attending large sporting events in Saudi Arabia and beyond. Through the combination of interactive indoor navigation, real-time crowd monitoring, AI-assisted guidance, and administrative management tools, the application addresses the complete lifecycle of the stadium visitor journey from arrival to seat.

The project successfully demonstrates the integration of modern mobile development technologies — Flutter for cross-platform UI, Supabase for scalable backend services, Flutter Riverpod for reactive state management, and Google Gemini for conversational AI — into a cohesive, production-quality application.

Key achievements of the project include: a fully functional cross-platform mobile application; a custom stadium map renderer using Flutter CustomPainter with pixel-accurate waypoint navigation; a dynamic rerouting engine that responds to real-time crowd data; an AI navigation chatbot with contextual ticket and crowd awareness; and a comprehensive administrative dashboard with gate management, crowd monitoring, and notification broadcasting.

Future development directions include: AI-based predictive crowd modelling using historical event data, integration with official Saudi ticketing APIs (Webook), live GPS-assisted outdoor navigation to the stadium, multi-stadium support beyond Alinma Stadium, and accessibility enhancements. The architecture has been specifically designed to accommodate stadium expansion with minimal refactoring, positioning WEJHA for commercial deployment across the Saudi Pro League venue network.

REFERENCES

- Flutter Team. *Flutter Documentation: Cross-Platform UI Toolkit*. Google LLC, <https://flutter.dev/docs>
- Supabase Inc. *Supabase Documentation: Open Source Firebase Alternative*. <https://supabase.com/docs>
- PostgreSQL Global Development Group. *PostgreSQL 15 Documentation*. <https://www.postgresql.org/docs/15/>
- Google LLC. *Gemini API Documentation: Generative AI for Developers*. <https://ai.google.dev/docs>
- Riverpod. *Flutter Riverpod: State Management Library Documentation*. <https://riverpod.dev/docs>
- GoRouter. *GoRouter: Declarative Routing for Flutter*. Flutter Team. https://pub.dev/packages/go_router
- Supabase Inc. *Supabase Realtime: Multiplayer and Real-Time Features*. <https://supabase.com/docs/guides/realtime>
- Supabase Inc. *Row Level Security in PostgreSQL with Supabase*. <https://supabase.com/docs/guides/database/postgres/row-level-security>
- Al-Rashidi, A., & Al-Ghamdi, S. *Indoor Navigation Systems for Large Public Venues: A Survey*. Journal of Location Based Services.
- Statista Research Department. *Football Attendance and Stadium Capacity in Saudi Pro League*. Statista, 2023. <https://www.statista.com>
- Saudi Arabian Football Federation. *King Abdullah Sports City (Alinma Stadium) Official Specifications*. SAFF. <https://www.saff.com.sa>
- Dart Team. *Dart Programming Language Documentation*. Google LLC. <https://dart.dev/guides>

APPENDIX-A: USER MANUAL (ONLY FOR PROJECT 2)

Prerequisites

Make sure the following are installed on your machine before running the app:

- Flutter SDK (version 3.0 or above) — <https://flutter.dev/docs/get-started/install>
- Dart SDK (included with Flutter)
- Android Studio or Xcode (for running on emulator or physical device)
- A connected Android or iOS device, or an emulator/simulator running

Step 1: Install Dependencies

Inside the project folder, run:

```
flutter pub get
```

This downloads all required packages listed in pubspec.yaml.

Step 2: Configure Supabase

Open the file lib/core/constants/app_constants.dart and fill in your Supabase project credentials:

```
const supabaseUrl =  
'https://canary.discord.com/channels/1242516055805530185/12  
42516056279617629/1512872625263874088';  
  
const supabaseAnonKey =  
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJIUzI6ImZubmRydmlncGtyeHhrY3V4cGN0liwicm9sZSI6ImFub24iLCJpYXQiOiE3Nzg1Mjc3ODQsImV4cCI6ImJA5NDEwMzc4NH0.Fvesg3o5BV0RFwel0F7IO1KDqjxQ6OuVA9myxhi_jGU';
```

You can find these values in your Supabase project dashboard under Settings > API.

Step 3: Configure Gemini API Key

Open the file lib/core/constants/app_constants.dart and add your Gemini API key:

```
const geminiApiKey = 'AlzaSyCtgfWju9EZujvmNDemWL4tqgy18wti_Kc';  
';
```

You can get a key from <https://ai.google.dev>

Step 4: Run the App

Connect your device or start an emulator, then run:

```
flutter run
```